



SAPIENZA
UNIVERSITÀ DI ROMA

Foundations of Data Science

Appunti del corso

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Corso di Laurea Magistrale in Computer Science

Docente

Prof. Indro Spinelli

Appunti di

Alessio Cavedal

GitHub: <https://github.com/AlessioCavedal>

Anno Accademico 2026/2027

Indice

1	Data science e machine learning	1
1.1	Che cos'è la data science	1
1.2	Intelligenza artificiale e machine learning	2
1.3	I tre paradigmi dell'apprendimento	3
1.4	Supervised learning	4
1.5	Unsupervised learning	7
1.6	Reinforcement learning	8
2	I dati come vettori	10
2.1	I dati nel supervised learning	10
2.2	Tipi di dato ed encoding	11
2.3	Il dataset in termini formali	11
2.4	I punti come vettori	13
2.5	Spanning set e basi	14
2.6	La base standard e le basi ortonormali	14
2.7	Basi qualsiasi: encoding e calcolo dei pesi	15
2.8	Sottospazi e proiezioni	17
3	Principal Component Analysis	19
3.1	Le trasformazioni lineari	19
3.2	Encoding, decoding ed errore di ricostruzione	19
3.3	Imparare la base	20
3.4	Dalla ricostruzione alla varianza	21
3.5	La matrice di covarianza	22
3.6	Le componenti principali	23
3.7	A che cosa serve	24
3.8	Un esempio svolto	24
3.9	Quando funziona: i manifold	25

Capitolo 1

Data science e machine learning

Questo capitolo inquadra il corso: che cos'è la data science, perché al suo centro c'è il machine learning, e quali sono i tre grandi paradigmi dell'apprendimento automatico, ciascuno con i suoi esempi.

1.1 Che cos'è la data science

Definizione (data science). La **data science** è un campo interdisciplinare che estrae dai dati conoscenza e indicazioni concrete su cui agire (*actionable insights*). Comprende la raccolta, la pulizia, la gestione e l'analisi di dati strutturati e non strutturati, con tecniche che vengono dalla statistica, dalla matematica, dall'informatica e dalla conoscenza del dominio applicativo.

Un progetto di data science non è un'analisi isolata, ma un ciclo che si ripete (Figura 1.1). Le fasi sono cinque:

- *Business Understanding*: si definiscono gli obiettivi di business e li si traduce in obiettivi di machine learning;
- *Data Engineering*: si raccolgono e si verificano i dati, li si pulisce e li si visualizza, li si arricchisce con nuovi esempi (*data augmentation*) e li si standardizza;
- *Model Engineering*: si sceglie il modello di machine learning, lo si addestra e lo si valuta;
- *Deployment*: il modello va in produzione, e si verifica che gli utenti lo accettino e riescano a usarlo;
- *Monitoring*: se ne controlla l'efficacia nel tempo, lo si riaddestra quando serve e si raccolgono dati nuovi, che riportano all'inizio del ciclo.

Esempio (un servizio in abbonamento). Un servizio di streaming perde troppi abbonati. Nella fase di *business understanding* l'obiettivo «perdere meno clienti» diventa un problema di machine learning: prevedere, per ogni abbonato, se disdirà nel prossimo mese, così da offrirgli uno sconto in tempo. Nel *data engineering* si raccolgono i dati utili (da quanto è abbonato, quanto guarda, se

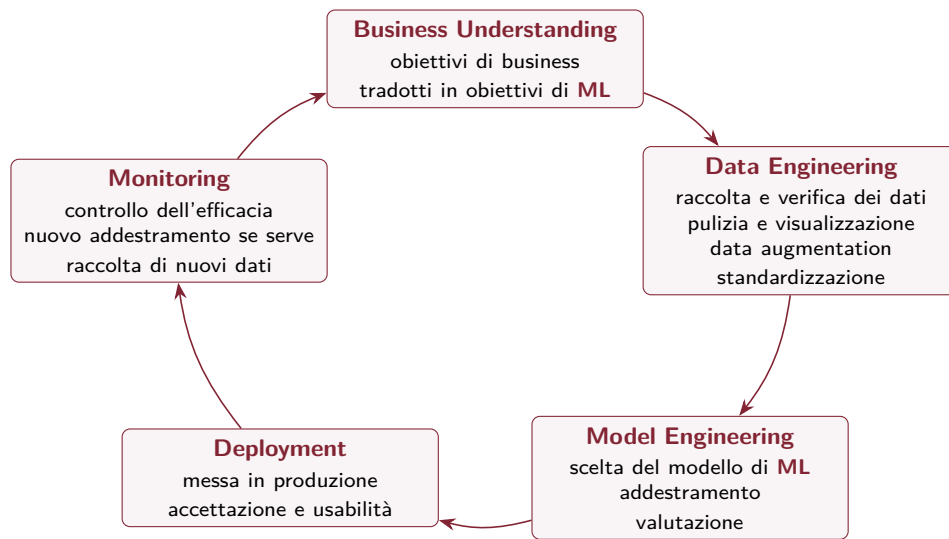


Figura 1.1. Il ciclo di vita di un progetto di data science. Il machine learning (ML) entra sia nella formulazione degli obiettivi sia nella scelta del modello.

ha contattato l'assistenza), si eliminano errori e duplicati e si portano tutti i dati sulla stessa scala. Nel *model engineering* si sceglie un modello, lo si addestra sui clienti del passato, di cui si sa chi ha disdetto, e si misura quanto bene prevede. Con il *deployment* le previsioni arrivano ogni giorno al reparto marketing, in una forma che sappia usare. Infine, con il *monitoring*, si controlla che le previsioni restino buone: se le abitudini degli utenti cambiano, si raccolgono dati nuovi e si riaddestra il modello, e il ciclo ricomincia.

Il machine learning compare in due punti chiave del ciclo: gli obiettivi vanno formulati come problemi di machine learning, e il cuore del lavoro tecnico è scegliere e addestrare un modello. Per questo il corso si concentra sul machine learning.

1.2 Intelligenza artificiale e machine learning

L'**intelligenza artificiale** (*Artificial Intelligence*, AI) è il campo dell'informatica che si occupa di costruire agenti capaci di svolgere compiti che di solito richiedono l'intelligenza umana, come imparare, ragionare e risolvere problemi. Il **machine learning** (ML, apprendimento automatico) ne studia un aspetto preciso: l'apprendimento dall'esperienza. La definizione classica è di Tom Mitchell.

Definizione (machine learning). Il machine learning studia gli algoritmi che migliorano la propria *prestazione* P in un *compito* T grazie all'*esperienza* E . Un problema di apprendimento ben definito è dato dalla terna $\langle P, T, E \rangle$.

La definizione obbliga a dire con precisione che cosa si vuole imparare (T), come si misura il successo (P) e da quali dati si impara (E). La Tabella 1.1 riporta quattro esempi.

Compito T	Prestazione P	Esperienza E
giocare a dama	percentuale di partite vinte contro un avversario qualsiasi	partite di allenamento giocate contro sé stesso
riconoscere parole scritte a mano	percentuale di parole classificate correttamente	database di immagini di parole scritte a mano, etichettate da persone
guidare su un'autostrada a quattro corsie con sensori visivi	distanza media percorsa prima di un errore, giudicato da una persona	sequenza di immagini e di comandi di sterzo registrati osservando un guidatore
classificare le email come spam o legittime	percentuale di email classificate correttamente	database di email, alcune etichettate a mano

Tabella 1.1. Quattro problemi di apprendimento descritti con la terna di Mitchell.

Il machine learning serve perché molti compiti, come riconoscere persone e oggetti in un'immagine o capire il parlato e il linguaggio, sono troppo complessi per scriverne le regole a mano. L'approccio del machine learning consiste allora nel costruire algoritmi che imparano da soli, a partire dai dati o dall'esperienza.

Machine learning e statistica. Machine learning e statistica hanno molto in comune: entrambi cercano regolarità (*pattern*) nei dati, usano analisi, probabilità e algebra lineare, e condividono molti algoritmi. Cambia l'enfasi. La statistica punta sull'inferenza: trarre conclusioni rigorose e interpretabili sul fenomeno che ha prodotto i dati. Il machine learning punta sulla previsione, sulla scalabilità e sull'autonomia del sistema. In breve: stessa matematica, obiettivi diversi.

1.3 I tre paradigmi dell'apprendimento

Il machine learning si divide in tre paradigmi, a seconda dei dati da cui si impara:

- nel **supervised learning** (apprendimento supervisionato) si hanno esempi etichettati del comportamento corretto, cioè coppie formate da un input e dall'output desiderato. Per esempio, migliaia di email ciascuna con l'indicazione «spam» o «legittima»: il sistema impara a dare l'etichetta giusta alle email nuove;
- nell'**unsupervised learning** (apprendimento non supervisionato) non ci sono etichette, e si cercano nei dati strutture o regolarità «interessanti». Per esempio, dagli acquisti dei clienti di un negozio si scopre che esistono alcuni gruppi con abitudini simili, senza che nessuno abbia detto prima quali siano;
- nel **reinforcement learning** (apprendimento per rinforzo) un sistema che apprende, l'**agente**, interagisce con il mondo e impara ad agire in modo razionale, massimizzando un segnale scalare di ricompensa (*reward*). Per esempio, un programma che gioca a scacchi: nessuno gli dice qual è la mossa giusta, ma vincere o perdere pezzi e partite gli indica se sta giocando bene.

La Tabella 1.2 riassume che cosa riceve in ingresso e che cosa produce ciascun paradigma.

Paradigma	Input	Output
<i>Supervised</i>	esempi di input insieme all'output desiderato	un modello che predice l'output per un input nuovo
<i>Unsupervised</i>	esempi di dati, senza output	una rappresentazione della struttura dei dati
<i>Reinforcement</i>	una sequenza di interazioni con un ambiente	una <i>policy</i> , cioè una strategia che svolge il compito desiderato

Tabella 1.2. Input e output dei tre paradigmi dell'apprendimento.

1.4 Supervised learning

Nel supervised learning si vuole una funzione che porti dagli input agli output, e la si impara da esempi di coppie input-output. È un rovesciamento rispetto alla programmazione tradizionale (Figura 1.2): lì il programmatore scrive il programma e il computer lo applica ai dati per produrre l'output; qui si forniscono al computer i dati insieme agli output attesi, ed è il computer a ricavarne il «programma», cioè il modello.

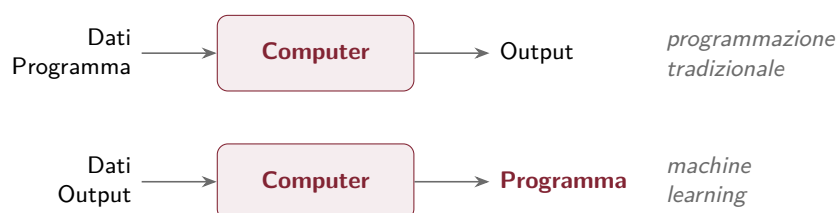


Figura 1.2. Nella programmazione tradizionale il programma è un ingresso; nel machine learning è il risultato, ricavato dai dati e dagli output attesi.

1.4.1 Dall'input reale all'output reale

Tutti gli esempi di supervised learning seguono lo stesso schema (Figura 1.3). L'input del mondo reale, che siano le caratteristiche di una casa, un testo o un'immagine, viene trasformato in un vettore di numeri, l'input del modello. Il modello produce un altro vettore, l'output del modello, che si interpreta di nuovo nel mondo reale: un prezzo, una classe, una frase. Un modello lavora soltanto su vettori di numeri: come si rappresentano i dati in questa forma è l'argomento del Capitolo 2.

1.4.2 Tipi di problema

In base all'output si distinguono:

- la **regressione**, in cui l'output è un numero reale: è **univariata** se l'output è uno solo, **multivariata** se sono più d'uno;

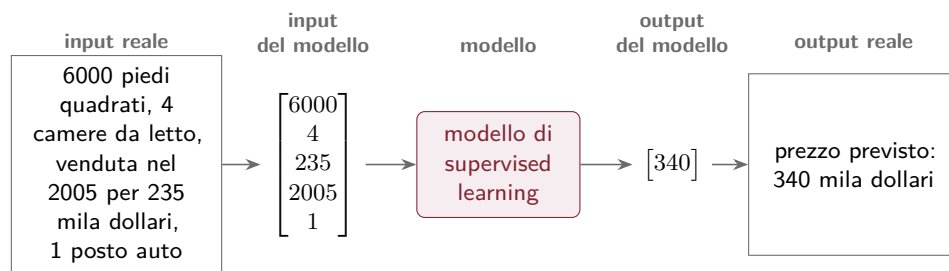


Figura 1.3. Lo schema comune agli esempi di supervised learning, nella stima del prezzo di una casa: l'input reale diventa un vettore, il modello produce un altro vettore, e questo si traduce nell'output reale.

- la **classificazione**, in cui l'output è una classe scelta in un insieme discreto: è **binaria** se le classi sono due, **multiclasse** se sono più di due;
- la **classificazione binaria multivariata**, con molti output, ciascuno con due classi possibili: per esempio decidere, pixel per pixel, se un'immagine mostra un oggetto o lo sfondo.

Nei problemi di classificazione il modello di solito non restituisce direttamente la classe, ma un valore per ciascuna classe: nella classificazione di una recensione, per esempio, 0,02 per «positiva» e 0,98 per «negativa». Si sceglie la classe con il valore più alto.

La Tabella 1.3 raccoglie gli esempi della lezione, con il tipo di problema e l'architettura di rete neurale usata. Le architetture sono solo citate: per ora conta che compiti diversi richiedono modelli diversi.

Gli ultimi tre esempi non rientrano nei tipi elencati sopra: l'output non è un numero né una classe, ma un oggetto strutturato (una frase, un'immagine), e le risposte valide possono essere molte.

1.4.3 Che cos'è un modello

Un modello di supervised learning è, in sostanza, un'equazione che lega l'input all'output. Nell'esempio della Figura 1.4 l'input è l'età di un bambino e l'output la sua altezza: per un bambino di 10 anni il modello prevede 139 cm. Addestrare il modello significa cercare, in una *famiglia* di equazioni possibili, quella che si adatta bene ai dati di addestramento (*training data*), i punti del grafico.

Per capire che cosa sia una famiglia di equazioni, si pensi alle rette $\text{altezza} = a + b \cdot \text{età}$. Ogni coppia di numeri a e b , i **parametri**, dà una retta diversa: con $a = 80$ e $b = 6$ un bambino di 10 anni risulterebbe alto 140 cm, con $a = 50$ e $b = 2$ appena 70 cm. Addestrare il modello vuol dire scegliere i valori di a e b che fanno passare la retta il più vicino possibile ai punti, cioè che sbagliano di meno sui bambini di cui si conosce già l'altezza. Una volta scelti i parametri, il modello si usa sui bambini nuovi. Le rette però sono una famiglia povera: la crescita non è lineare (rallenta dopo l'adolescenza), e nessuna retta segue bene i punti della figura. Per questo la curva disegnata viene da una famiglia più flessibile.

Le reti neurali profonde (*deep neural networks*) sono semplicemente una famiglia di equazioni molto flessibile, capace di descrivere relazioni complicatissime tra input

Compito	Input → output	Tipo di problema	Architettura
regressione	caratteristiche di una casa → prezzo	regressione univariata	rete <i>fully connected</i>
<i>graph regression</i>	molecola → temperatura di congelamento e di ebollizione	regressione multivariata	<i>graph neural network</i>
classificazione di testi	recensione di un ristorante → positiva o negativa	classificazione binaria	<i>transformer</i>
genere musicale	brano audio → genere (classica, jazz, pop...)	classificazione multiclasse	rete ricorrente (RNN)
<i>image classification</i>	immagine → oggetto raffigurato (bicicletta, ape...)	classificazione multiclasse	rete convoluzionale
<i>image segmentation</i>	immagine → per ogni pixel, oggetto o sfondo	classificazione binaria multivariata	<i>encoder-decoder</i> convoluzionale
<i>depth estimation</i>	immagine → profondità di ogni pixel	regressione multivariata	<i>encoder-decoder</i> convoluzionale
<i>pose estimation</i>	immagine di una persona → posizione delle articolazioni	regressione multivariata	<i>encoder-decoder</i> convoluzionale
traduzione	testo in inglese → testo in francese	output strutturato	—
<i>image captioning</i>	immagine → frase che la descrive	output strutturato	—
generazione di immagini	descrizione testuale → immagine	output strutturato	—

Tabella 1.3. Esempi di supervised learning, con il tipo di problema e l'architettura di rete.

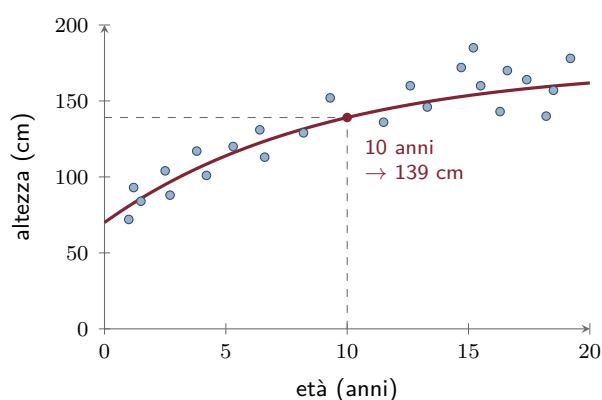


Figura 1.4. Un modello come equazione: la curva lega l'età all'altezza ed è scelta perché si adatta ai dati di addestramento (i punti). Per un bambino di 10 anni prevede 139 cm.

e output. L'idea è la stessa delle rette, ma invece di due parametri ce ne sono milioni, e per questo la famiglia contiene funzioni adatte anche a compiti come riconoscere un oggetto in un'immagine. Adattarle ai dati è ciò che si chiama **deep learning**.

1.4.4 Che cosa hanno in comune gli esempi

Gli esempi della Tabella 1.3 hanno tre tratti comuni:

- la relazione tra input e output è molto complessa;
- a volte le risposte valide sono molte: una frase si traduce in più modi, e una descrizione corrisponde a infinite immagini;
- gli output, e talvolta gli input, obbediscono a delle regole: il linguaggio rispetta una grammatica, e anche le immagini naturali hanno le loro «regole», perché non ogni combinazione di pixel sembra una fotografia.

Da qui un'idea: imparare la «grammatica» dei dati da esempi senza etichetta, che si possono raccogliere in quantità enormi proprio perché nessuno deve etichettarli. Sapendo bene quali output sono plausibili, il compito supervisionato diventa più facile. È uno dei motivi per cui l'apprendimento non supervisionato è così importante.

1.5 Unsupervised learning

Nell'unsupervised learning si impara qualcosa su un dataset senza etichette. I compiti tipici sono:

- il **clustering**, cioè raggruppare i dati simili. Un esempio è DeepCluster (Caron et al., 2018), che raggruppa le immagini con caratteristiche visive simili senza sapere che cosa raffigurano;
- trovare gli **outlier**, i dati anomali;
- generare nuovi esempi, simili a quelli del dataset;
- ricostruire i dati mancanti.

Una classe importante è quella dei **modelli generativi**, capaci di creare nuovi esempi: da migliaia di foto di gatti, per esempio, imparano a produrre foto di gatti che non esistono. Ne sono un esempio le *generative adversarial networks* (GAN). Al loro interno, i **modelli generativi probabilistici** (*probabilistic generative models*, PGM) imparano esplicitamente una distribuzione di probabilità sui dati: tra questi i *variational autoencoder* (VAE), i *normalizing flow* e i *diffusion model*. I tre insieme sono uno dentro l'altro (Figura 1.5).

Molti modelli generativi si basano sulle **variabili latenti** (Figura 1.6): si estraggono a caso pochi numeri da distribuzioni semplici, per esempio gaussiane, e un modello di deep learning li trasforma in un dato osservato ad alta dimensione, come un'immagine. L'idea è che dati complessi, fatti di moltissimi numeri, dipendano da pochi fattori nascosti. Una foto di un volto, per esempio, è fatta di centinaia di migliaia di pixel, ma ciò che la determina è poco: la persona, la posa, la luce, l'espressione. Le variabili latenti giocano il ruolo di questi fattori, anche se il modello li impara da solo e non hanno per forza un significato così chiaro. Per generare un

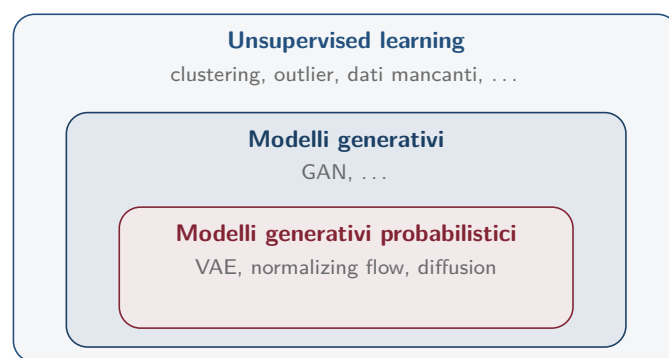


Figura 1.5. I modelli generativi sono una parte dell'unsupervised learning, e i modelli generativi probabilistici una parte dei modelli generativi.

esempio nuovo basta estrarre nuovi valori delle variabili latenti e farli passare nel modello.

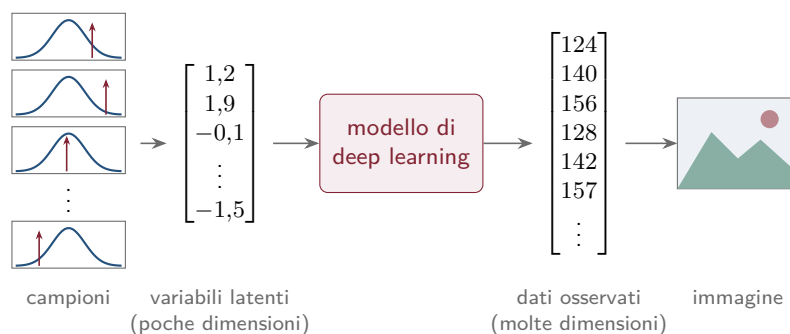


Figura 1.6. Variabili latenti: pochi valori estratti da distribuzioni semplici diventano, attraverso un modello di deep learning, un dato ad alta dimensione come un'immagine.

1.6 Reinforcement learning

Nel reinforcement learning ci sono un insieme di **stati**, un insieme di **azioni** e un insieme di **ricompense**. L'obiettivo è scegliere le azioni in modo da cambiare lo stato e ricevere ricompense. A differenza degli altri due paradigmi non si riceve nessun dataset: l'agente deve esplorare l'ambiente e raccogliere i dati da sé, man mano che agisce (Figura 1.7).

Esempio (gli scacchi). Gli stati sono le configurazioni valide della scacchiera e le azioni, in un dato momento, sono le mosse consentite. Le ricompense sono positive quando si mangia un pezzo dell'avversario e negative quando se ne perde uno.

Il problema è difficile per tre motivi:

- è **stocastico**: la stessa mossa ripetuta due volte può avere esiti diversi, perché l'avversario può rispondere in un altro modo, e anche le ricompense sono

incerte (l'avversario può mangiare un pezzo oppure no);

- c'è il **temporal credit assignment problem** (attribuzione temporale del merito): quando arriva una ricompensa, non è chiaro se sia merito dell'ultima mossa o di buone scelte tattiche fatte molto prima;
- c'è il **exploration-exploitation trade-off** (compromesso tra esplorazione e sfruttamento): se si è trovata una buona apertura, conviene usare sempre quella o provarne altre, nella speranza di trovarne una migliore?

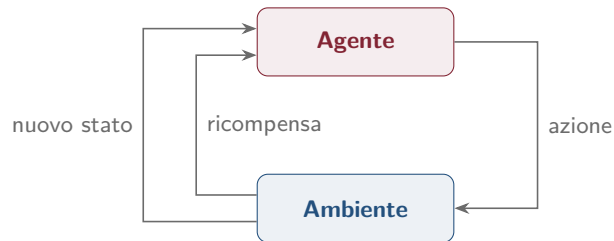


Figura 1.7. Il ciclo del reinforcement learning: l'agente sceglie un'azione, l'ambiente risponde con un nuovo stato e una ricompensa.

Capitolo 2

I dati come vettori

Nel capitolo precedente si è visto che un modello lavora su vettori di numeri. Questo capitolo guarda da vicino i dati: come sono fatti nel supervised learning, come li si converte in numeri e come li si descrive in termini formali. Poi tratta ogni dato come un vettore di uno spazio vettoriale, e da qui arriva alle nozioni di base, encoding e proiezione.

2.1 I dati nel supervised learning

Un esempio classico, introdotto dallo statistico Ronald Fisher e usato spesso come banco di prova, è il dataset **IRIS**: 150 fiori di iris, 50 per ciascuna di tre specie (*setosa*, *versicolor* e *virginica*), di cui sono misurate in centimetri lunghezza e larghezza del sepal e del petalo. Il compito è riconoscere la specie a partire dalle quattro misure.

In generale i dati del supervised learning sono misure di diversi aspetti di n oggetti, raccolte in una tabella con una riga per oggetto (Tabella 2.1). Le colonne sono di due tipi:

- il **target** è la variabile di output, l'obiettivo della previsione: qui la specie;
- le **feature** sono le proprietà che descrivono l'oggetto: qui le quattro misure.

Si assume che tra feature e target ci sia una relazione, e si vuole scoprire la regola predittiva che la descrive.

Feature $\mathbf{x}$				Target y
<i>Sepal.Length</i>	<i>Sepal.Width</i>	<i>Petal.Length</i>	<i>Petal.Width</i>	<i>Species</i>
4,3	3,0	1,1	0,1	setosa
5,0	3,3	1,4	0,2	setosa
7,7	3,8	6,7	2,2	virginica
5,5	2,5	4,0	1,3	versicolor

Tabella 2.1. Quattro righe del dataset IRIS: le misure in centimetri sono le feature, la specie è il target.

Le voci della colonna del target si chiamano **label** (etichette), e si distinguono due tipi di dati:

- i **dati etichettati** (*labeled data*), in cui il target è osservato;
- i **dati non etichettati** (*unlabeled data*), in cui il target è sconosciuto.

Il supervised learning impara dai primi; l'unsupervised learning (Sezione 1.5) lavora sui secondi.

2.2 Tipi di dato ed encoding

Feature e target possono essere di tipi diversi:

- **numerici**, con valori in $\mathbb{R}$;
- **interi**, con valori in $\mathbb{Z}$;
- **categorici**, con valori in un insieme finito di categorie $\{C_1, \dots, C_g\}$;
- **binari**, con valori in $\{0, 1\}$.

Il tipo del target decide il tipo di problema (Sezione 1.4): un target numerico dà un problema di regressione, un target categorico un problema di classificazione. IRIS, con la specie come target, è un problema di classificazione con tre classi.

Quasi tutti gli algoritmi di apprendimento lavorano solo con feature numeriche, salvo qualche eccezione. Le feature degli altri tipi vanno quindi convertite in numeri: bisogna scegliere, o costruire, un **encoding** (codifica) adatto.

Definizione (one-hot encoding). Con l'**encoding one-hot**, una variabile categorica con k categorie mutuamente esclusive passa da valore scalare a vettore binario di dimensione k , con un solo elemento uguale a 1, nella posizione della categoria, e 0 altrove. La categoria i -esima diventa

$$\mathbf{e}_i = (0, \dots, 0, \underbrace{1}_{\text{posizione } i}, 0, \dots, 0)^\top, \quad i \in \{1, \dots, k\}.$$

Esempio (le specie di IRIS). Con le specie nell'ordine setosa, versicolor, virginica, l'encoding one-hot è

$$\text{setosa} \mapsto (1, 0, 0)^\top, \quad \text{versicolor} \mapsto (0, 1, 0)^\top, \quad \text{virginica} \mapsto (0, 0, 1)^\top.$$

Se invece le categorie hanno un ordine naturale, come «basso», «medio» e «alto», si usa un encoding che lo rispecchi, per esempio una sequenza di interi: 1, 2, 3. Per categorie senza ordine, come le specie, gli interi sarebbero fuorvianti, perché introdurrebbero un ordine e delle distanze che non esistono (versicolor non sta «tra» setosa e virginica); i vettori one-hot invece sono tutti alla stessa distanza l'uno dall'altro.

2.3 Il dataset in termini formali

La tabella dei dati ha anche una scrittura matematica, che conviene vedere subito su IRIS. Ogni riga è un'**osservazione**, cioè una coppia $(\mathbf{x}^{(i)}, y^{(i)})$: $\mathbf{x}^{(i)}$ è il vettore

delle feature dell' i -esimo fiore, $y^{(i)}$ la sua specie. Per la prima riga della Tabella 2.1

$$\mathbf{x}^{(1)} = (4,3; 3,0; 1,1; 0,1), \quad y^{(1)} = \text{setosa}.$$

L'indice tra parentesi in alto dice di quale osservazione si parla, quello in basso di quale feature: $x_3^{(1)} = 1,1$ è la terza feature (la lunghezza del petalo) del primo fiore. Con l'indice in basso da solo si indica un'intera colonna: $\mathbf{x}_1$ è il vettore delle 150 lunghezze del sepal, una per fiore.

Il **dataset** è l'insieme di tutte le osservazioni,

$$D = ((\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})),$$

con $n = 150$ per IRIS. I vettori $\mathbf{x}^{(i)}$ stanno nello **spazio degli input** X , di solito $X \subseteq \mathbb{R}^p$ quando le p feature sono numeriche (per IRIS $p = 4$); i target stanno nello **spazio degli output** Y , che per IRIS è l'insieme delle tre specie.

2.3.1 Il processo che genera i dati

I 150 fiori di IRIS non sono tutti i fiori di iris che esistono: sono un campione. Si immagina che dietro i dati ci sia un processo che li genera, descritto da una distribuzione di probabilità P_{XY} sulle coppie (input, target): dice quanto è probabile incontrare un fiore con certe misure e di una certa specie. Le variabili aleatorie che seguono questa distribuzione si indicano con le minuscole $\mathbf{x}$ e y .

Che serva una distribuzione, e non una semplice funzione da $\mathbf{x}$ a y , si capisce dai dati stessi: due fiori con le stesse misure possono essere di specie diverse, perché le specie si sovrappongono un po', e ogni misura ha un errore. Dato $\mathbf{x}$, quindi, la specie non è determinata con certezza: alcune specie sono solo più probabili di altre.

La **distribuzione vera** P_{XY} di solito è sconosciuta, e si vedono solo i campioni che ne sono usciti. Il supervised learning cerca di impararne la parte che serve per prevedere y a partire da $\mathbf{x}$.

Definizione (ipotesi i.i.d.). Si assume che le osservazioni siano estratte da P_{XY} in modo **i.i.d.** (*independent and identically distributed*, indipendenti e identicamente distribuite): tutte provengono dalla stessa distribuzione, e ciascuna non dipende dalle altre.

Per IRIS l'ipotesi è ragionevole se i fiori sono stati raccolti a caso: misurare un fiore non dice nulla sul successivo, e tutti vengono dalla stessa popolazione. Non vale invece per il prezzo di un'azione giorno per giorno, perché il prezzo di oggi dipende molto da quello di ieri: è una serie temporale. È quindi un'ipotesi forte e non sempre realistica; esistono scenari più complessi che ne fanno a meno, ma qui ci si limita al caso i.i.d.

2.4 I punti come vettori

Cambio di notazione. Da qui in avanti le slide, e con loro gli appunti, seguono la notazione del libro *Machine Learning Refined*:

- il dataset è formato da P punti $\mathbf{x}_1, \dots, \mathbf{x}_P \in \mathbb{R}^N$: P prende il posto di n , e N quello di p ;
- l'indice p scorre sui punti, l'indice n sulle dimensioni e sui vettori di una base;
- $\mathbf{x}_p$ è il p -esimo *punto* (una riga della tabella), non una colonna come $\mathbf{x}_j$ nella Sezione 2.3.

D'ora in poi si guarda solo agli input, ignorando il target. Si assume inoltre che il dataset sia **centrato** (*mean-centered*): a ogni punto si sottrae la media del dataset, dimensione per dimensione,

$$\bar{\mathbf{x}} = \frac{1}{P} \sum_{p=1}^P \mathbf{x}_p, \quad \mathbf{x}_p \leftarrow \mathbf{x}_p - \bar{\mathbf{x}},$$

così che i dati si dispongano attorno all'origine. In IRIS, per esempio, la lunghezza media del sepal è circa 5,8 cm: dopo la centratura un fiore con il sepal di 4,3 cm ha valore $-1,5$, che si legge come «1,5 cm più corto della media». È un'operazione semplice e del tutto reversibile: basta sommare di nuovo la media.

Ogni osservazione del dataset è un punto dello spazio degli input. Uno **spazio vettoriale** è un insieme di oggetti, i **vettori**, che si possono sommare tra loro e moltiplicare per numeri, gli **scalari**; $\mathbb{R}^N$ ne è l'esempio tipico. Un punto di $\mathbb{R}^N$ si può disegnare come un punto o come una freccia che parte dall'origine (Figura 2.1): sono due modi di vedere lo stesso oggetto, il primo più comodo per guardare un dataset, il secondo per ragionare su somme e multipli.

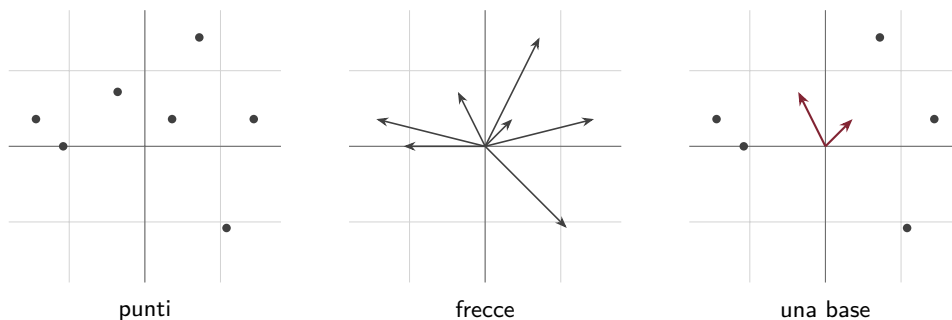


Figura 2.1. Lo stesso dataset in $\mathbb{R}^2$ visto come punti (a sinistra) e come frecce che partono dall'origine (al centro). A destra due soli vettori, che formano una base: ogni altro punto è una loro combinazione lineare.

2.5 Spanning set e basi

Definizione (spanning set). Uno **spanning set** (insieme di generatori) è un insieme di vettori che, combinati con somme e prodotti per scalari, producono *ogni* vettore dello spazio. Si dice che i vettori **generano** (*span*) lo spazio.

Per esempio, in $\mathbb{R}^2$ i vettori $\mathbf{e}_1 = (1, 0)^\top$ e $\mathbf{e}_2 = (0, 1)^\top$ generano il piano, perché ogni punto (x, y) si scrive come $x\mathbf{e}_1 + y\mathbf{e}_2$.

Definizione (base). Una **base** è uno spanning set minimale: il più piccolo insieme di vettori che genera ancora lo spazio. Nessun vettore deve essere ridondante, cioè i vettori devono essere **linearmente indipendenti**: nessuno si può ottenere come combinazione lineare degli altri.

In $\mathbb{R}^2$ la coppia $\{\mathbf{e}_1, \mathbf{e}_2\}$ è una base. Se le si aggiunge $\mathbf{e}_1 + \mathbf{e}_2 = (1, 1)^\top$, i tre vettori generano ancora il piano, ma il terzo è ridondante, perché è la somma degli altri due. La conseguenza pratica è che i modi di scrivere un punto diventano infiniti: $(2, 1)$ è $2\mathbf{e}_1 + \mathbf{e}_2$, ma anche $\mathbf{e}_1 + (\mathbf{e}_1 + \mathbf{e}_2)$, e così via. Con una base invece la scrittura è unica, ed è questo che la rende utile: ogni punto ha un solo insieme di «coordinate».

Perché una base $\mathbf{c}_1, \dots, \mathbf{c}_N$ possa rappresentare perfettamente tutti i P punti del dataset, deve vivere nello stesso spazio $\mathbb{R}^N$ dei dati, e per ogni punto deve esistere un insieme di **pesi** con cui la combinazione lineare dei vettori della base restituisce il punto:

$$\sum_{n=1}^N \mathbf{c}_n w_{n,p} = \mathbf{x}_p, \quad p = 1, \dots, P. \quad (2.1)$$

Il peso $w_{n,p}$ dice quanto del vettore $\mathbf{c}_n$ serve per ricostruire il punto $\mathbf{x}_p$.

2.6 La base standard e le basi ortonormali

La base più semplice è la **base standard**: il vettore $\mathbf{c}_n$ ha tutti zeri tranne un 1 nella posizione n , cioè $\mathbf{c}_n = (0, \dots, 0, 1, 0, \dots, 0)^\top$. Le sue proprietà chiave sono due: vettori diversi sono ortogonali, e ogni vettore ha norma unitaria,

$$\mathbf{c}_n^\top \mathbf{c}_m = 0 \quad \text{per } n \neq m, \quad \|\mathbf{c}_n\|_2^2 = 1 \quad \text{per } n = 1, \dots, N.$$

Rappresentare un punto sulla base standard è banale: i pesi coincidono con il punto stesso, $\mathbf{w}_p = \mathbf{x}_p$, cioè ogni peso è il valore della coordinata corrispondente (Figura 2.2, a sinistra). I vettori one-hot della Sezione 2.2 sono proprio i vettori della base standard di $\mathbb{R}^k$.

Impilando i vettori della base come colonne di una matrice $\mathbf{C}$, le due proprietà si scrivono in forma compatta:

$$\mathbf{C}^\top \mathbf{C} = \mathbf{I}_{N \times N}.$$

L'elemento di posto (n, m) di $\mathbf{C}^\top \mathbf{C}$ è infatti il prodotto scalare $\mathbf{c}_n^\top \mathbf{c}_m$, che vale 1 sulla diagonale e 0 altrove. Un insieme di vettori con questa proprietà si dice **ortonormale**.

La base standard non è l'unica base ortonormale: ruotando gli assi se ne ottengono altre, e con tutte i pesi si trovano senza fatica. Il motivo è geometrico. Il prodotto scalare $\mathbf{c}_n^\top \mathbf{x}_p$ tra un vettore di lunghezza 1 e un punto è la lunghezza dell'«ombra» del punto sulla retta di $\mathbf{c}_n$, cioè quanto il punto avanza nella direzione di $\mathbf{c}_n$. Se gli assi sono perpendicolari tra loro, queste ombre non si disturbano a vicenda, e sono esattamente i pesi:

$$w_{n,p} = \mathbf{c}_n^\top \mathbf{x}_p, \quad \text{cioè} \quad \mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p.$$

Lo si verifica anche con l'algebra: la rappresentazione (2.1) si scrive $\mathbf{C}\mathbf{w}_p = \mathbf{x}_p$, e moltiplicando a sinistra per $\mathbf{C}^\top$, con $\mathbf{C}^\top \mathbf{C} = \mathbf{I}$, resta proprio $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$. Non c'è nessun sistema di equazioni da risolvere: il vettore dei pesi, chiamato **encoding** del punto, si ottiene con un prodotto scalare per ciascun vettore della base. Rimettendo l'encoding nella rappresentazione si ritrova il punto di partenza, $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p = \mathbf{x}_p$: $\mathbf{C}^\top$ codifica, $\mathbf{C}$ decodifica.

Esempio (assi ruotati di 45°). I vettori $\mathbf{c}_1 = (1, 1)^\top / \sqrt{2}$ e $\mathbf{c}_2 = (-1, 1)^\top / \sqrt{2}$ hanno lunghezza 1 e sono perpendicolari ($\mathbf{c}_1^\top \mathbf{c}_2 = (-1 + 1)/2 = 0$): sono gli assi standard ruotati di 45°. Per il punto $\mathbf{x} = (3, 1)^\top$ i pesi sono due prodotti scalari:

$$w_1 = \mathbf{c}_1^\top \mathbf{x} = \frac{3+1}{\sqrt{2}} = 2\sqrt{2}, \quad w_2 = \mathbf{c}_2^\top \mathbf{x} = \frac{-3+1}{\sqrt{2}} = -\sqrt{2}.$$

La verifica: $2\sqrt{2}\mathbf{c}_1 - \sqrt{2}\mathbf{c}_2 = (2, 2)^\top + (1, -1)^\top = (3, 1)^\top$. Il punto avanza molto lungo $\mathbf{c}_1$ e un po' all'indietro lungo $\mathbf{c}_2$.

2.7 Basi qualsiasi: encoding e calcolo dei pesi

Se la base non è quella standard, il vettore dei pesi $\mathbf{w}_p$ è diverso dal punto $\mathbf{x}_p$: è la sua nuova rappresentazione, il suo encoding, nel **transformed feature space** (spazio delle feature trasformato), lo spazio i cui assi coordinati sono i vettori della base. Descrive lo stesso punto, ma «misurato» lungo direzioni diverse.

Si prenda per esempio, in $N = 2$ dimensioni, la base

$$\mathbf{c}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{c}_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

I due vettori sono linearmente indipendenti, perché puntano in direzioni diverse, quindi qualunque punto del piano si rappresenta come loro combinazione lineare:

$$w_{1,p}\mathbf{c}_1 + w_{2,p}\mathbf{c}_2 = \mathbf{x}_p,$$

con pesi $w_{1,p}$ e $w_{2,p}$ che sono unici per ogni punto $\mathbf{x}_p$. La Figura 2.2 confronta le due rappresentazioni dello stesso punto: con la base standard i pesi sono le coordinate del punto, con la base $\mathbf{c}_1, \mathbf{c}_2$ sono diversi, e nello spazio trasformato il punto ha per coordinate proprio quei pesi.

Esempio (calcolo dei pesi). Per il punto $\mathbf{x} = (5, 5)^\top$ (omettendo l'indice p) la condizione $w_1 \mathbf{c}_1 + w_2 \mathbf{c}_2 = \mathbf{x}$ è il sistema

$$\begin{cases} 2w_1 + w_2 = 5 \\ w_1 + 2w_2 = 5 \end{cases}$$

che ha soluzione $w_1 = w_2 = 5/3 \approx 1,667$. Allo stesso modo il punto $(-3, 0)^\top$ ha encoding $(-2, 1)^\top$: infatti $-2\mathbf{c}_1 + \mathbf{c}_2 = (-4 + 1, -2 + 2)^\top = (-3, 0)^\top$. La scorciatoia delle basi ortonormali qui non funziona: $\mathbf{C}^\top \mathbf{x} = (15, 15)^\top$ non è l'encoding, perché $\mathbf{c}_1$ e $\mathbf{c}_2$ non sono ortogonali ($\mathbf{c}_1^\top \mathbf{c}_2 = 4$).

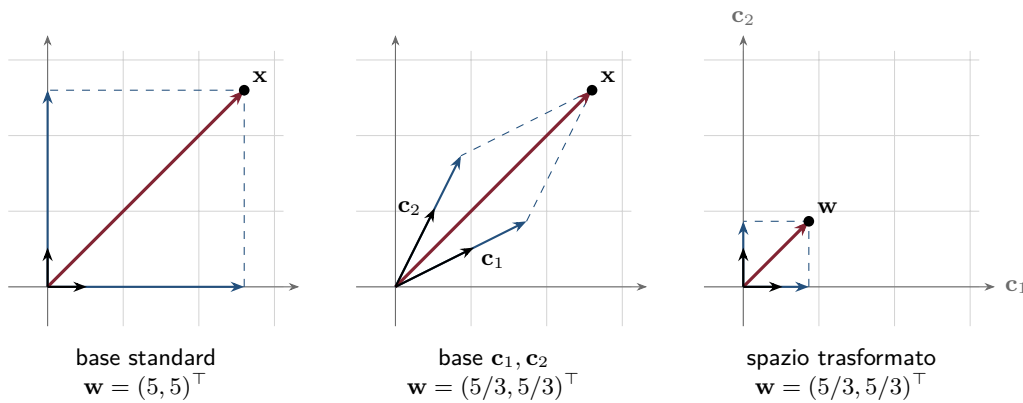


Figura 2.2. Il punto $\mathbf{x} = (5, 5)^\top$ sulla base standard (a sinistra) e sulla base $\mathbf{c}_1 = (2, 1)^\top$, $\mathbf{c}_2 = (1, 2)^\top$ (al centro): in nero i vettori della base, in blu i vettori scalati dai pesi, in rosso la loro somma, che è il punto. A destra l'encoding $\mathbf{w}$ nello spazio trasformato, i cui assi corrispondono a $\mathbf{c}_1$ e $\mathbf{c}_2$.

2.7.1 Trovare i pesi

Nell'esempio il sistema era piccolo e si risolveva a mano. In generale conviene vedere il calcolo dei pesi come un problema di ottimizzazione: si cercano i pesi con cui la combinazione dei vettori della base, $\mathbf{C}\mathbf{w}_p$, cade il più vicino possibile al punto $\mathbf{x}_p$. ($\mathbf{C}$ è la matrice $N \times N$ che ha per colonne i vettori della base.) La distanza si misura con il quadrato della norma, quindi per ogni punto si minimizza

$$h(\mathbf{w}_p) = \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2.$$

Quando la base genera tutto lo spazio il minimo è zero, perché esiste una combinazione che riproduce esattamente il punto. Scriverlo come minimizzazione però è utile, perché lo stesso problema ha senso anche quando la riproduzione esatta è impossibile (Sezione 2.8). Annullando il gradiente di h , che vale $2\mathbf{C}^\top(\mathbf{C}\mathbf{w}_p - \mathbf{x}_p)$, si ottiene un sistema lineare simmetrico, le **equazioni normali**:

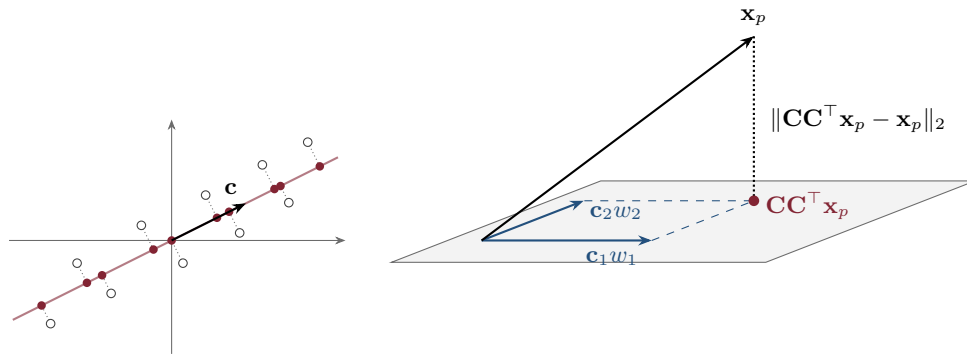
$$\mathbf{C}^\top \mathbf{C} \mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p.$$

Con una base ortonormale $\mathbf{C}^\top \mathbf{C} = \mathbf{I}$, e si ritrova $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$. Per trovare i pesi di tutti i punti insieme si somma (o si media) il costo dei singoli punti, $\frac{1}{P} \sum_p \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2$. Finché la base è fissata non cambia nulla, perché ogni punto ha i suoi pesi; la forma media diventerà importante nel Capitolo 3, quando anche la base andrà imparata.

2.8 Sottospazi e proiezioni

Ricapitolando, perché una base possa rappresentare perfettamente qualunque punto di uno spazio a N dimensioni servono due condizioni:

1. i vettori sono linearmente indipendenti, cioè puntano in direzioni diverse;
2. i vettori sono almeno N .



$N = 2, K = 1$: proiezione su una retta

$N = 3, K = 2$: proiezione su un piano

Figura 2.3. A sinistra, punti di $\mathbb{R}^2$ (cerchi vuoti) proiettati sulla retta generata da un solo vettore $\mathbf{c}$: i punti pieni sono le proiezioni. A destra, un punto $\mathbf{x}_p$ di $\mathbb{R}^3$ proiettato sul piano generato da due vettori ortonormali $\mathbf{c}_1$ e $\mathbf{c}_2$: il punto proiettato è $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p = \mathbf{c}_1 w_1 + \mathbf{c}_2 w_2$, e il segmento punteggiato è l'errore di approssimazione.

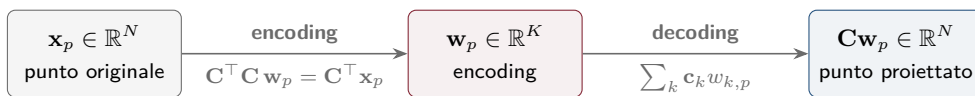


Figura 2.4. Encoding e decoding: il punto originale, fatto di N numeri, diventa un encoding di K numeri, da cui il decoding ricostruisce il punto proiettato, di nuovo in $\mathbb{R}^N$.

Che cosa succede se si rinuncia alla seconda, e i vettori sono solo $K < N$? Non si può più rappresentare perfettamente un insieme generico di P punti: in N dimensioni, K vettori generano al massimo un **sottospazio** di dimensione K . Per esempio, con $N = 3$:

- $K = 2$ vettori generano al più un piano, un iperpiano di $\mathbb{R}^3$;
- $K = 1$ vettore genera soltanto una retta.

In entrambi i casi non si raggiungono tutti i punti dello spazio, ma solo quelli di un sottospazio di dimensione più bassa. Un sottospazio passa sempre per l'origine: anche per questo conviene lavorare con dati centrati.

Se il punto non sta nel sottospazio non lo si può riprodurre esattamente, ma ci si può avvicinare il più possibile: è proprio il problema di minimo della sezione precedente, ora con un minimo diverso da zero. I pesi si trovano quindi con le stesse equazioni normali, $\mathbf{C}^\top \mathbf{C} \mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$, con una sola differenza: ora $\mathbf{C}$ ha dimensione $N \times K$, perché contiene solo i K vettori della base, e quindi $\mathbf{w}_p \in \mathbb{R}^K$.

Questo procedimento si chiama **proiezione**, perché il punto più vicino del sottospazio è quello che si ottiene «facendo cadere» il punto perpendicolarmente sul sottospazio (Figura 2.3): qualunque altro punto del sottospazio è più lontano, come in un triangolo rettangolo l'ipotenusa è più lunga del cateto. Il vettore dei pesi $\mathbf{w}_p$ è l'encoding del punto nel sottospazio: K numeri invece di N . Il passo di **decoding** ricostruisce il punto combinando l'encoding con i K vettori della base: il risultato $\mathbf{C} \mathbf{w}_p$ è il **punto proiettato**, la migliore approssimazione del punto originale all'interno del sottospazio (Figura 2.4).

Se la base di K elementi è ortonormale, l'encoding è di nuovo fatto di prodotti scalari, $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$, e il decoding dà

$$\mathbf{C} \mathbf{C}^\top \mathbf{x}_p \approx \mathbf{x}_p.$$

Al posto dell'uguaglianza della Sezione 2.6 ora c'è un'approssimazione: il punto proiettato coincide con $\mathbf{x}_p$ solo se $\mathbf{x}_p$ sta già nel sottospazio. L'errore commesso è la distanza tra il punto e la sua proiezione, $\|\mathbf{C} \mathbf{C}^\top \mathbf{x}_p - \mathbf{x}_p\|_2$.

Esempio (proiezione su una retta). Si riprenda il punto $\mathbf{x} = (3, 1)^\top$ dell'esempio sugli assi ruotati, ma tenendo solo il primo vettore, $\mathbf{c}_1 = (1, 1)^\top / \sqrt{2}$: ora $K = 1$, $N = 2$, e il sottospazio è la retta $y = x$. L'encoding è un solo numero, $w_1 = \mathbf{c}_1^\top \mathbf{x} = 2\sqrt{2}$. Il decoding dà il punto proiettato $w_1 \mathbf{c}_1 = (2, 2)^\top$, il punto della retta più vicino a $(3, 1)$. L'errore è la distanza tra i due, $\|(3, 1) - (2, 2)\|_2 = \|(1, -1)\|_2 = \sqrt{2}$, e non è un caso che sia uguale (in valore assoluto) al peso $w_2 = -\sqrt{2}$ che si è buttato via: togliere un asse di una base ortonormale significa perdere esattamente la parte del punto lungo quell'asse.

Capitolo 3

Principal Component Analysis

Nel Capitolo 2 la base era sempre data, e si trattava solo di trovare i pesi con cui rappresentare i punti. Qui si fa il passo successivo: si impara anche la base, scegliendo le direzioni che descrivono meglio i dati. Il risultato è la **Principal Component Analysis** (PCA, analisi delle componenti principali), il metodo più importante di unsupervised learning per rappresentare i dati con meno numeri.

3.1 Le trasformazioni lineari

Una matrice A di dimensione $m \times n$ trasforma ogni vettore $\mathbf{x} \in \mathbb{R}^n$ in un vettore $A\mathbf{x} \in \mathbb{R}^m$. Conviene pensarla come una trasformazione dell'intero spazio, e per capire che cosa fa basta guardare dove manda i vettori della base standard: le colonne di A sono proprio le immagini di $\mathbf{e}_1, \mathbf{e}_2, \dots$, e ogni altro vettore, che è una loro combinazione, si trasforma di conseguenza. A seconda della struttura di A l'effetto è diverso (Figura 3.1):

- **scala**: se A è diagonale, ogni asse viene allungato o accorciato del fattore che sta sulla diagonale;
- **rotazione** o **riflessione**: se A è ortonormale ($A^\top A = \mathbf{I}$), lo spazio ruota o si specchia senza deformarsi, e lunghezze e angoli restano gli stessi;
- **shear** (deformazione di taglio): gli elementi fuori dalla diagonale inclinano le direzioni, e un quadrato diventa un parallelogramma;
- **proiezione**: se il rango di A (la dimensione dello spazio generato dalle sue colonne) è $k < n$, lo spazio viene schiacciato su un sottospazio di dimensione k . Punti diversi possono finire nello stesso punto, quindi una parte dell'informazione si perde.

Nella PCA contano soprattutto due di questi casi: le rotazioni, che cambiano gli assi senza cambiare i dati, e le proiezioni, che riducono le dimensioni.

3.2 Encoding, decoding ed errore di ricostruzione

Si riprendano le basi ortonormali della Sezione 2.6, con i vettori della base nelle colonne della matrice $\mathbf{C}$.

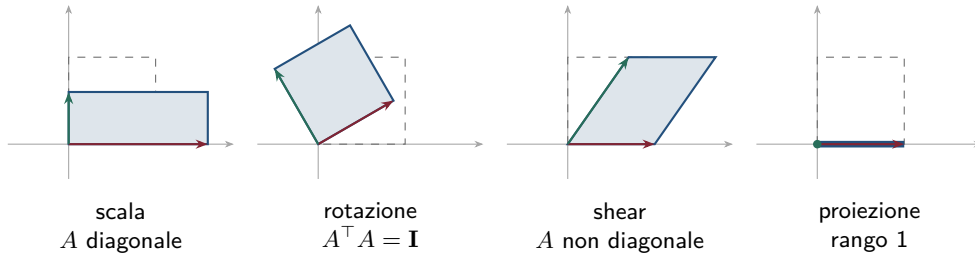


Figura 3.1. Il quadrato unitario (tratteggiato) e la sua immagine sotto quattro trasformazioni lineari. Le frecce sono le immagini di $\mathbf{e}_1$ (in rosso) e di $\mathbf{e}_2$ (in verde), cioè le colonne di A ; nella proiezione $\mathbf{e}_2$ finisce nell'origine.

- Se i vettori sono N , tanti quante le dimensioni, l'encoding $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$ è una rotazione: il punto non cambia, cambiano solo gli assi lungo cui lo si misura. Il decoding $\mathbf{C}\mathbf{w}_p$ restituisce esattamente il punto, quindi $\mathbf{C}^\top$ codifica e $\mathbf{C}$ decodifica, senza perdite.
- Se i vettori sono $K < N$, l'encoding si calcola allo stesso modo, ma il decoding dà solo un'approssimazione, $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p \approx \mathbf{x}_p$: è la proiezione ortogonale del punto sul sottospazio generato dalla base (Sezione 2.8). Ciò che il punto ha fuori dal sottospazio si perde.

Quanto si perde lo misura l'**errore di ricostruzione**. Per ogni punto si vuole che la ricostruzione $\mathbf{C}\mathbf{w}_p$ sia il più vicino possibile al punto, e sull'intero dataset si prende la media degli errori:

$$\frac{1}{P} \sum_{p=1}^P \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2.$$

Con una base ortonormale i pesi migliori sono $\mathbf{w}_p = \mathbf{C}^\top \mathbf{x}_p$, e sostituendoli l'errore dipende solo dalla base:

$$g(\mathbf{C}) = \frac{1}{P} \sum_{p=1}^P \|\mathbf{C}\mathbf{C}^\top \mathbf{x}_p - \mathbf{x}_p\|_2^2.$$

Ogni addendo è il quadrato della distanza perpendicolare tra il punto e il sottospazio (il segmento punteggiato della Figura 2.3). Una volta fissata la base, questo errore non si può ridurre: i pesi sono già i migliori possibili. L'unico modo di abbassarlo è cambiare il sottospazio.

3.3 Imparare la base

Finora la base $\mathbf{C}$ era data e si imparavano solo i pesi. La PCA impara anche la base: tra tutti i sottospazi di dimensione K cerca quello che passa più vicino ai dati, cioè che rende minimo l'errore di ricostruzione. In due dimensioni con $K = 1$, per esempio, la domanda è: quale retta per l'origine sta più vicina a una nuvola di punti? A occhio è la retta lungo cui la nuvola è allungata, e il resto del capitolo mostra che è proprio così.

Il problema da risolvere è

$$g(\mathbf{w}_1, \dots, \mathbf{w}_P, \mathbf{C}) = \frac{1}{P} \sum_{p=1}^P \|\mathbf{C}\mathbf{w}_p - \mathbf{x}_p\|_2^2,$$

da minimizzare sia nei pesi sia nella base. Questa funzione di costo si chiama **autoencoder** (lineare), perché si imparano insieme l'encoding, i pesi $\mathbf{w}_p$, e il decoding, la ricostruzione $\mathbf{C}\mathbf{w}_p$: il sistema impara a ricostruire il proprio input facendolo passare per una «strettoia» di soli K numeri. Si può minimizzare con uno qualunque dei metodi di ottimizzazione standard.

Se $\mathbf{C}$ è ortonormale, i pesi ottimi sono $\mathbf{C}^\top \mathbf{x}_p$ e il costo si riduce a $g(\mathbf{C})$ della sezione precedente. Non serve nemmeno imporre l'ortonormalità come vincolo: ciò che conta per la ricostruzione è il sottospazio generato dalla base, e il sottospazio migliore si può sempre descrivere con una base ortonormale. Da qui in avanti, quindi, la base si considera ortonormale.

3.4 Dalla ricostruzione alla varianza

L'osservazione chiave è geometrica. Il punto $\mathbf{x}_p$, la sua proiezione $\mathbf{C}\mathbf{C}^\top \mathbf{x}_p$ e l'errore formano un triangolo rettangolo (Figura 2.3), quindi per il teorema di Pitagora

$$\|\mathbf{C}\mathbf{C}^\top \mathbf{x}_p - \mathbf{x}_p\|_2^2 = \|\mathbf{x}_p\|_2^2 - \|\mathbf{C}^\top \mathbf{x}_p\|_2^2.$$

(La lunghezza della proiezione è uguale a quella dell'encoding, $\|\mathbf{C}^\top \mathbf{x}_p\|$, perché $\mathbf{C}$ ha colonne ortonormali e non cambia le lunghezze. La stessa uguaglianza si ottiene sviluppando il quadrato e usando $\mathbf{C}^\top \mathbf{C} = \mathbf{I}$.) La lunghezza del punto non dipende dalla base, quindi *minimizzare l'errore equivale a massimizzare la lunghezza della proiezione*: la base migliore è quella su cui i punti proiettano l'«ombra» più lunga.

Scrivendo la norma dell'encoding componente per componente, $\|\mathbf{C}^\top \mathbf{x}_p\|_2^2 = \sum_{n=1}^K (\mathbf{c}_n^\top \mathbf{x}_p)^2$, il problema diventa

$$\text{massimizzare} \quad \frac{1}{P} \sum_{p=1}^P \sum_{n=1}^K (\mathbf{c}_n^\top \mathbf{x}_p)^2.$$

Non ci sono termini che mescolano vettori diversi della base: ogni $\mathbf{c}_n$ dà il suo contributo separatamente, e i vettori si possono scegliere uno alla volta (restando perpendicolari tra loro).

Per il primo vettore la quantità da massimizzare è

$$h(\mathbf{c}_1) = \frac{1}{P} \sum_{p=1}^P (\mathbf{c}_1^\top \mathbf{x}_p)^2.$$

Il numero $\mathbf{c}_1^\top \mathbf{x}_p$ è la coordinata del punto lungo la direzione $\mathbf{c}_1$, che ha lunghezza 1. Poiché i dati sono centrati, queste coordinate hanno media zero, e la media dei loro quadrati è proprio la loro **varianza**: $h(\mathbf{c}_1)$ è la varianza dei dati lungo la direzione $\mathbf{c}_1$, cioè quanto la nuvola si allarga in quella direzione. Il primo vettore della base va quindi scelto lungo la direzione in cui i dati variano di più, come suggeriva l'intuizione.

3.5 La matrice di covarianza

Per trovare quella direzione conviene raccogliere i dati in una matrice $\mathbf{X}$, di dimensione $N \times P$, che ha per colonne i punti $\mathbf{x}_1, \dots, \mathbf{x}_P$. La varianza lungo $\mathbf{c}_1$ si riscrive allora come

$$h(\mathbf{c}_1) = \frac{1}{P} \mathbf{c}_1^\top \mathbf{X} \mathbf{X}^\top \mathbf{c}_1 = \mathbf{c}_1^\top \Sigma \mathbf{c}_1, \quad \text{con} \quad \Sigma = \frac{1}{P} \mathbf{X} \mathbf{X}^\top.$$

La matrice Σ , di dimensione $N \times N$, è la **matrice di covarianza** empirica, e riassume come variano le feature, da sole e a coppie:

- sulla diagonale, Σ_{ii} è la varianza della feature i : quanto quella feature si allarga attorno alla sua media;
- fuori dalla diagonale, Σ_{ij} è la **covarianza** tra le feature i e j : è positiva se le due tendono a crescere insieme, negativa se quando una cresce l'altra tende a calare, vicina a zero se non c'è una relazione lineare tra le due.

In IRIS, per esempio, la lunghezza e la larghezza del petalo hanno una covarianza positiva: i petali lunghi tendono anche a essere larghi.

3.5.1 Autovettori e autovalori

Geometricamente, Σ descrive la forma della nuvola di punti: in quali direzioni è orientata e quanto è allungata in ciascuna, come un'ellisse (in più dimensioni, un ellissoide) adattata ai dati (Figura 3.2). Le direzioni degli assi di questa ellisse sono gli **autovettori** di Σ , cioè i vettori $\mathbf{v}$ che Σ si limita ad allungare, senza cambiarne la direzione:

$$\Sigma \mathbf{v}_n = \lambda_n \mathbf{v}_n.$$

Il fattore λ_n è l'**autovalore** corrispondente, e dice quanta varianza hanno i dati lungo $\mathbf{v}_n$. (Nelle slide gli autovalori sono indicati a volte con λ_n , a volte con d_n .)

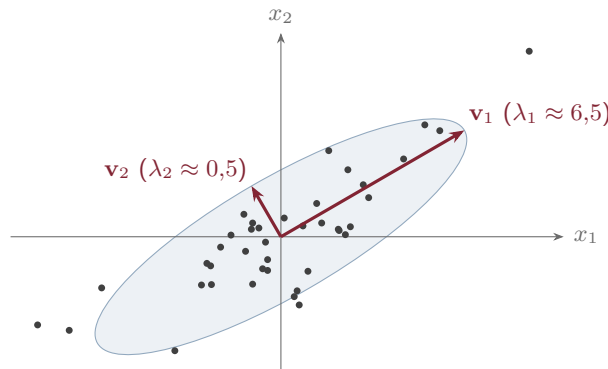


Figura 3.2. La matrice di covarianza come forma della nuvola di punti. Gli autovettori $\mathbf{v}_1$ e $\mathbf{v}_2$ indicano gli assi dell'ellisse; le frecce sono lunghe quanto i semiassi, cioè due deviazioni standard ($2\sqrt{\lambda_n}$) lungo ciascuna direzione.

La covarianza è una matrice simmetrica ($\Sigma_{ij} = \Sigma_{ji}$), e le matrici simmetriche reali hanno due proprietà che qui fanno tutto il lavoro:

- gli autovalori sono reali, e per una matrice di covarianza anche non negativi (una varianza non può essere negativa);

- gli autovettori di autovalori diversi sono ortogonali, e si possono prendere di lunghezza 1: formano quindi una base ortonormale, cioè un nuovo sistema di assi cartesiani, ruotato rispetto a quello di partenza.

Nel sistema di assi degli autovettori la trasformazione Σ è semplicissima: allunga ciascun asse $\mathbf{v}_n$ del fattore λ_n e basta. È ciò che dice la **decomposizione spettrale**

$$\Sigma = \mathbf{V}\Lambda\mathbf{V}^\top,$$

dove $\mathbf{V} = [\mathbf{v}_1 \cdots \mathbf{v}_N]$ ha gli autovettori in colonna (ed è ortonormale, $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$) e Λ è la matrice diagonale degli autovalori. Letta da destra a sinistra: $\mathbf{V}^\top$ porta un vettore nel sistema degli autovettori (è un encoding), Λ scala ogni coordinata del suo autovalore, $\mathbf{V}$ riporta il risultato negli assi originali.

3.6 Le componenti principali

Resta da trovare il vettore $\mathbf{c}_1$, di lunghezza 1, che rende massima la varianza $\mathbf{c}_1^\top \Sigma \mathbf{c}_1$. La risposta è l'autovettore $\mathbf{v}_1$ con l'autovalore più grande, e il massimo vale λ_1 :

$$\mathbf{c}_1 = \mathbf{v}_1, \quad h(\mathbf{v}_1) = \lambda_1.$$

Il motivo si vede scrivendo un vettore qualunque di lunghezza 1 nella base degli autovettori, $\mathbf{c} = a_1\mathbf{v}_1 + \cdots + a_N\mathbf{v}_N$, con $a_1^2 + \cdots + a_N^2 = 1$. Poiché Σ allunga ogni $\mathbf{v}_n$ del suo λ_n , e gli autovettori sono ortonormali, si ottiene

$$\mathbf{c}^\top \Sigma \mathbf{c} = \lambda_1 a_1^2 + \lambda_2 a_2^2 + \cdots + \lambda_N a_N^2,$$

cioè una media degli autovalori con pesi a_n^2 che sommano a 1. Una media non può superare il valore più grande tra quelli mediati, e lo raggiunge solo mettendo tutto il peso su di esso: $a_1 = 1$ e gli altri zero, cioè $\mathbf{c} = \mathbf{v}_1$. Questa direzione è la **prima componente principale** del dataset.

Il secondo vettore va cercato allo stesso modo, $h(\mathbf{c}_2) = \mathbf{c}_2^\top \Sigma \mathbf{c}_2$, ma perpendicolare al primo, $\mathbf{c}_1^\top \mathbf{c}_2 = 0$. Essere perpendicolare a $\mathbf{v}_1$ vuol dire avere $a_1 = 0$: la media pesata si fa solo sugli autovalori restanti, e il massimo è il secondo autovalore, λ_2 , raggiunto per $\mathbf{c}_2 = \mathbf{v}_2$, la **seconda componente principale**. Si procede così fino a K .

Il ragionamento si chiude così: minimizzare l'errore di ricostruzione dell'autoencoder lineare equivale a massimizzare la varianza lungo direzioni successive e perpendicolari. La soluzione sono i primi K autovettori della matrice di covarianza, le **componenti principali**, e ciascun autovalore dice quanta varianza cattura la sua componente. La PCA classica, quindi, è proprio il minimo globale dell'autoencoder lineare. La Figura 3.3 mostra il risultato su un dataset in due dimensioni con $K = 1$.

Due conseguenze tornano utili nei conti. La varianza totale dei dati, la somma delle varianze delle singole feature, è uguale alla somma di tutti gli autovalori. Tenendo le prime K componenti, la frazione di varianza conservata, la **varianza spiegata**, è

$$\frac{\lambda_1 + \cdots + \lambda_K}{\lambda_1 + \cdots + \lambda_N},$$

e per Pitagora l'errore di ricostruzione medio è proprio la varianza scartata, $\lambda_{K+1} + \cdots + \lambda_N$. Nel dataset della figura $\lambda_1 \approx 6,51$ e $\lambda_2 \approx 0,50$: la prima componente da sola spiega circa il 93% della varianza.

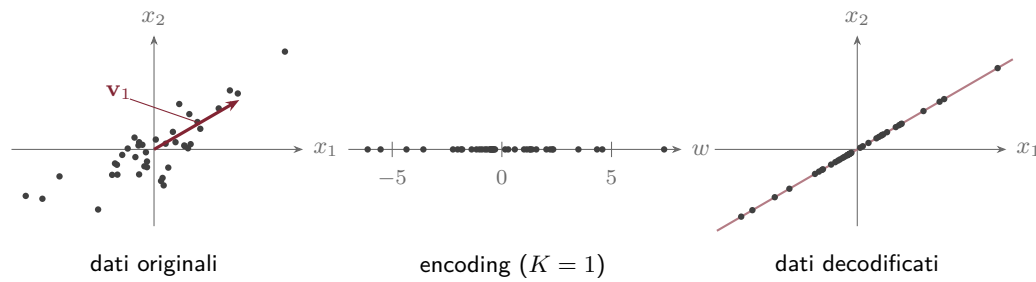


Figura 3.3. PCA con $K = 1$ sul dataset della Figura 3.2. A sinistra i dati e la prima componente principale $\mathbf{v}_1$; al centro l'encoding, un solo numero per punto ($w_p = \mathbf{v}_1^\top \mathbf{x}_p$); a destra i punti decodificati, $w_p \mathbf{v}_1$, che stanno tutti sulla retta di $\mathbf{v}_1$.

3.7 A che cosa serve

- **Riduzione della dimensionalità.** La PCA trova nuovi assi ortogonali, le componenti principali, allineati con le direzioni di massima varianza. Tenendo solo le prime $K < N$ si passa da N a K numeri per punto, comprimendo i dati ma conservando quanta più variabilità (informazione) possibile.
- **Feature engineering.** Le componenti principali sono feature nuove e *scorrelate*: nel sistema degli autovettori la covarianza è la matrice diagonale Λ , quindi le covarianze tra feature diverse sono zero. Usarle al posto delle feature originali può semplificare il lavoro dei modelli successivi, migliorarne le prestazioni e ridurre l'*overfitting*.
- **Unsupervised learning.** La PCA non usa etichette: impara la struttura direttamente dai dati. È il metodo fondamentale di *representation learning* non supervisionato: impara una base nuova che spiega le correlazioni nei dati, e così rivela le direzioni di forte correlazione e le ridondanze (feature che dicono quasi la stessa cosa).

3.8 Un esempio svolto

Esempio (PCA a mano su tre punti). Si considerino i tre punti di $\mathbb{R}^2$, non centrati,

$$\mathbf{x}_1 = (2, 1)^\top, \quad \mathbf{x}_2 = (0, 0)^\top, \quad \mathbf{x}_3 = (1, -1)^\top.$$

1. **Centrata.** La media è $\bar{\mathbf{x}} = (1, 0)^\top$, e i punti centrati sono $\tilde{\mathbf{x}}_1 = (1, 1)^\top$, $\tilde{\mathbf{x}}_2 = (-1, 0)^\top$, $\tilde{\mathbf{x}}_3 = (0, -1)^\top$. (Le slide li chiamano $\mathbf{c}_p$, che qui creerebbe confusione con i vettori della base.)
2. **Covarianza.** Con $P = 3$, $\Sigma = \frac{1}{3} \sum_p \tilde{\mathbf{x}}_p \tilde{\mathbf{x}}_p^\top = \frac{1}{3} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Sulla diagonale le due varianze, entrambe $2/3$; fuori la covarianza, $1/3$.
3. **Autovalori e autovettori.** Gli autovalori sono $\lambda_1 = 1$ e $\lambda_2 = 1/3$, con autovettori $\mathbf{v}_1 = (1, 1)^\top / \sqrt{2}$ e $\mathbf{v}_2 = (1, -1)^\top / \sqrt{2}$. Si verifica subito:

$$\Sigma(1,1)^\top = \frac{1}{3}(3,3)^\top = (1,1)^\top \text{ e } \Sigma(1,-1)^\top = \frac{1}{3}(1,-1)^\top.$$

4. *Componenti principali.* La prima è $\mathbf{v}_1$, la direzione della bisettrice; la seconda è $\mathbf{v}_2$, perpendicolare.
5. *Encoding.* Proiettando i punti centrati su $\mathbf{v}_1$ si ottengono gli score $z_p = \mathbf{v}_1^\top \tilde{\mathbf{x}}_p$: $z_1 = \sqrt{2}$, $z_2 = -1/\sqrt{2}$, $z_3 = -1/\sqrt{2}$.
6. *Ricostruzione ed errore.* I punti ricostruiti sono $z_p \mathbf{v}_1$: $(1,1)$, che coincide con $\tilde{\mathbf{x}}_1$, poi $(-\frac{1}{2}, -\frac{1}{2})$ due volte. Gli errori quadratici sono 0, $\frac{1}{2}$ e $\frac{1}{2}$, e la loro media è $1/3$: esattamente λ_2 , la varianza scartata.
7. *Varianza spiegata.* La prima componente spiega $\lambda_1/(\lambda_1 + \lambda_2) = 1/(4/3) = 75\%$ della varianza.
8. *Segno della covarianza.* La covarianza tra le due coordinate è positiva: quando la prima è sopra la media, anche la seconda tende a esserlo. Per questo la direzione di massima varianza è la bisettrice $(1,1)$, e non $(1,-1)$.

3.9 Quando funziona: i manifold

Dati ad alta dimensione si possono spesso rappresentare con molte meno dimensioni. Succede quando stanno vicino a una *varietà lineare* dello spazio, cioè a un sottospazio: se lo si trova, si possono proiettare i dati su di esso e descriverli con le sue coordinate senza perdere molta informazione. È esattamente ciò che fa la PCA.

Un **manifold** (varietà) è uno spazio topologico che, visto da vicino, assomiglia allo spazio euclideo, anche se nel complesso può essere curvo. L'esempio tipico è la circonferenza unitaria $x^2 + y^2 = 1$: nel complesso è una curva chiusa, ma un suo pezzo piccolo, visto da vicino, sembra un segmento.

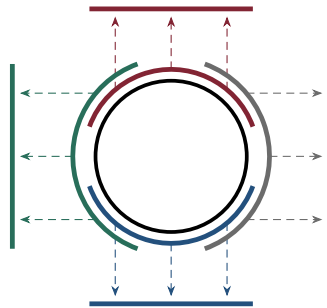


Figura 3.4. La circonferenza coperta da quattro chart: ogni arco aperto si proietta su un segmento, quelli in alto e in basso con la coordinata x , quelli a sinistra e a destra con la y . Gli archi si sovrappongono agli estremi.

Si prenda la metà superiore della circonferenza, i punti con $y > 0$. Ognuno è identificato in modo unico dalla sua coordinata x : la funzione $(x, y) \mapsto x$ manda l'arco sull'intervallo aperto $(-1, 1)$ in modo liscio e invertibile, perché dalla x si ricava il punto, $(x, \sqrt{1-x^2})$. Le funzioni che mettono in corrispondenza biunivoca una regione aperta della superficie con un sottoinsieme dello spazio euclideo si chiamano **chart** (carte). Una sola chart non basta per tutta la circonferenza (la x non distingue

un punto in alto da quello in basso), ma quattro, sugli archi superiore, inferiore, sinistro e destro, la coprono tutta (Figura 3.4).

Una chart si può vedere anche nel verso opposto, come funzione $\phi: \mathbb{R}^1 \rightarrow S \subset \mathbb{R}^2$ che dal parametro porta al punto della curva. Deve essere liscia e invertibile, con inversa anch'essa liscia (un **diffeomorfismo**: funzione e inversa sono derivabili con derivate continue). Il dominio di ϕ è lo **spazio dei parametri**, ed è euclideo; la sua immagine è l'**embedding**, la curva o superficie nello spazio. Un manifold, alla fine, è un'unione di chart. Con un manifold lineare basta la PCA; se il manifold è curvo, come la circonferenza, un sottospazio non lo segue bene.